

**Министерство науки и высшего образования Российской Федерации**  
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ**  
**УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ**  
**НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ**  
**ИТМО**  
**ITMO University**

**ЛАБОРАТОРНАЯ РАБОТА №1**

**По дисциплине:** Алгоритмы и структуры данных  
**Тема работы:** Бинарный поиск. Знакомство с графами.  
**Обучающиеся:** Казанцев Александр Сергеевич  
**Факультет:** Факультет прикладной информатики  
**Направление подготовки:** 11.04.02 Инфокоммуникационные технологии и системы связи  
**Группа:** К4112с  
**Руководитель:** Мусаев Андрей Александрович

Санкт-Петербург, 2026

## СОДЕРЖАНИЕ

|                                                            | Стр. |
|------------------------------------------------------------|------|
| <b>ВВЕДЕНИЕ</b>                                            | 3    |
| <b>1 ОСНОВНОЕ ЗАДАНИЕ</b>                                  | 4    |
| 1.1 Задание 1                                              | 4    |
| 1.2 Задание 2                                              | 5    |
| 1.2 Задание 3                                              | 8    |
| <b>2 LEETCODE</b>                                          | 10   |
| 2.1 Задача 1315. Sum of Nodes with Even-Valued Grandparent | 10   |
| 2.2 Задача 785. Is Graph Bipartite?                        | 10   |
| <b>ЗАКЛЮЧЕНИЕ</b>                                          | 10   |
| <b>СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ</b>                    | 11   |

## **ВВЕДЕНИЕ**

Целью данной работы является практическое изучение базовых алгоритмов поиска и знакомство со структурами графов. Представьте, что вы ищете номер телефона в бумажной книге, где миллион записей. Перебирать каждую строчку по очереди это линейный поиск, на который уйдут недели. В этой работе мы разберем бинарный поиск, который спасает от медленного кода и находит нужный элемент, каждый раз отсекая ровно половину списка. Также мы построим интерактивную систему угадывания студентов, обучающихся по данному предмету на нашем потоке. Это наглядно покажет, как компьютер принимает решения, двигаясь по развилкам признаков. Наконец, мы зафиксируем эти развилки в виде графа, то есть структуры, которая помогает связывать объекты между собой. Дополнительно в работе рассматривается решение практических задач с платформы LeetCode для закрепления теории графов.

# 1 ОСНОВНОЕ ЗАДАНИЕ

## 1.1 Задание 1

### Условие:

Написать программу для бинарного поиска. Результатом должно быть количество шагов, которое потребуется, чтобы найти загаданное число из массива.

### Объяснение решения:

Для решения этой задачи создадим функцию **binarySearch**. Она принимает на вход слайс чисел **nums** и число **target**, которое нам нужно найти, а возвращает количество шагов, потребовавшихся для успешного поиска.

Бинарный поиск работает исключительно с отсортированными. Поэтому первым делом, еще до вызова самой функции поиска, мы сортируем наш слайс в функции **main** с помощью стандартного инструмента **slices.Sort**.

Внутри функции поиска мы применяем метод двух указателей. Объявляем переменные **l** для начала слайса с индексом ноль и **r** для индекса самого последнего элемента. Также создаем переменную счетчика **count** для подсчета выполненных шагов.

С помощью цикла мы начинаем сужать область поиска в два раза на каждом круге. На каждой итерации сначала увеличиваем счетчик шагов на единицу, затем вычисляем индекс середины слайса и сравниваем число на этой позиции с нашей целью через конструкцию **switch**.

Если число в середине совпало с целью, то функция сразу возвращает накопленное количество шагов **count**. Если оно меньше цели, это означает, что наше число находится правее, и мы сдвигаем левую границу. В остальных случаях число находится левее, и мы сдвигаем в левую сторону правую границу.

Если границы в итоге пересеклись, а число так и не нашлось, функция возвращает минус один.

## Код программы на Golang:

```
1 package main
2
3 import (
4     "fmt"
5     "slices"
6 )
7
8 func binarySearch(slice []int, target int) int {
9     l int := 0
10    r int := len(slice) - 1
11    count int := 0
12
13    for l <= r {
14        count++
15        mid int := l + (r-l)/2
16        currNum int := slice[mid]
17
18        switch {
19            case currNum == target:
20                return count
21            case currNum < target:
22                l = mid + 1
23            default:
24                r = mid - 1
25        }
26    }
27
28    return -1
29 }
30
31 func main() {
32     testSlice []int := []int{9, 1, 5, 3, 9, 6}
33     target int := 6
34
35     slices.Sort(x: testSlice)
36
37     steps int := binarySearch(slice: testSlice, target)
38     fmt.Printf(format: "Количество шагов, чтобы найти число %d равно %d\n", a...: target
39 }
```

## 1.2 Задание 2

### Условие:

Для своей учебной группы составить словарь, который будет описывать характеристики каждого из студентов. Реализовать программу, которая по определенным характеристикам будет угадывать студента.

### Объяснение решения:

Для выполнения этого задания мы разработали поисковую экспертную систему, которая работает по принципу последовательного отсекаания неподходящих вариантов.

Сначала мы создадим структуру **Student**. Это шаблон, который описывает характеристики человека с помощью простых логических ответов **да** или **нет**. Мы выделили **семь** ключевых признаков на основе реальной анкеты потока: **пол, курение, цвет волос, обучение в бакалавриате ИТМО, увлечение спортом, наличие домашних животных** и конкретно **собаки**. Внутри функции **main** мы собрали базу данных **groupData** в виде среза, куда внесли анкеты студентов.

Затем программа запускает опрос в консоли. Мы используем инструмент **bufio.NewReader** для считывания ввода пользователя с клавиатуры. Специальная функция **strings.ToLower** автоматически переводит любой ввод в нижний регистр, чтобы ответы, написанные в любом регистре, обрабатывались компьютером абсолютно одинаково.

Чтобы сделать систему логичной, мы применили оптимизацию опроса. Программа сначала спрашивает пользователя о наличии домашних животных. Если пользователь отвечает **нет**, система автоматически понимает, что собаки у студента быть не может, и пропускает этот вопрос, экономя шаг обхода.

После завершения опроса программа запускает цикл **for range** и начинает сравнивать введенные параметры с анкетными данными каждого студента из базы. Если все выбранные параметры совпали, имя человека добавляется в список подходящих кандидатов **matched**.

В самом конце программа проверяет этот список. Если совпадение только одно, компьютер выводит имя угаданного студента. Если под описание подошли несколько ребят, система перечислит их через запятую. Если же пользователь выбрал комбинацию признаков, которой нет в базе, программа сообщит, что совпадений не найдено.

#### Код программы на Golang:

```
22 package main
21
20 import (
19     "bufio"
18     "fmt"
17     "os"
16     "strings"
15 )
14
13 type Student struct {
12     Name      string
11     IsMale    bool
10     IsSmoker  bool
9     IsBrunette bool
8     FromITMO  bool
7     IsSports  bool
6     HasPets   bool
5     HasDog    bool
4 }
```

```

22 func main() {
21   groupData []Student := []Student{
20     Student{Name: "Егорова Мария", IsMale: false, IsSmoker: false, IsBrunette: true, FromITMO:
19     Student{Name: "Фролова Кристина", IsMale: false, IsSmoker: false, IsBrunette: false, FromIT
18     Student{Name: "Костюченко Тимофей", IsMale: true, IsSmoker: false, IsBrunette: false, FromI
17     Student{Name: "Портнова Ксения", IsMale: false, IsSmoker: true, IsBrunette: false, FromITMO
16     Student{Name: "Кенжаев Рахим", IsMale: true, IsSmoker: false, IsBrunette: true, FromITMO: t
15     Student{Name: "Даньшин Семён", IsMale: true, IsSmoker: true, IsBrunette: true, FromITMO: tr
14     Student{Name: "Шихайло Сергей", IsMale: true, IsSmoker: true, IsBrunette: false, FromITMO:
13     Student{Name: "Бабушкин Александр", IsMale: true, IsSmoker: true, IsBrunette: false, FromIT
12     Student{Name: "Якунин Андрей", IsMale: true, IsSmoker: false, IsBrunette: false, FromITMO:
11     Student{Name: "Казанцев Александр", IsMale: true, IsSmoker: false, IsBrunette: false, FromI
10     Student{Name: "Юркин Александр", IsMale: true, IsSmoker: true, IsBrunette: true, FromITMO:
9     Student{Name: "Панкратова Анна", IsMale: false, IsSmoker: true, IsBrunette: false, FromITMO
8     Student{Name: "Колесникова Лариса", IsMale: false, IsSmoker: false, IsBrunette: false, From
7   }
6
5   reader *bufio.Reader := bufio.NewReader(rd: os.Stdin)
4   ask func(q string) bool := func(q string) bool {
3     fmt.Print(a...: q + " (да/нет): ")
2     text string, _ error := reader.ReadString(delim: '\n')
1     return strings.TrimSpace(s: strings.ToLower(s: text)) == "да"
43 }
1

```

```

22   fmt.Println(a...: "Система определения студентов")
21
20   male bool := ask(q: "Студент мужского пола?")
19   smoke bool := ask(q: "Студент курит?")
18   brunette bool := ask(q: "Студент является брюнетом?")
17   itmo bool := ask(q: "Студент учился в ИТМО в бакалавриате?")
16   sports bool := ask(q: "Студент занимается спортом?")
15   pets bool := ask(q: "У студента есть домашние животные?")
14
13   dog bool := false
12   if pets {
11     dog = ask(q: "Это собака?")
10   }
9
8   var matched []string
7   for _ int, s Student := range groupData {
6     if s.IsMale == male &&
5       s.IsSmoker == smoke &&
4       s.IsBrunette == brunette &&
3       s.FromITMO == itmo &&
2       s.IsSports == sports &&
1       s.HasPets == pets &&
67      s.HasDog == dog {
1     matched = append(matched, s.Name)
2   }
3 }

```

```

9
8   fmt.Println(a...: "\nРезультат поиска:")
7   if len(matched) == 1 {
6     fmt.Printf(format: "Вы загадали студента: %s\n", a...: matched[0])
5   } else if len(matched) > 1 {
4     fmt.Printf(format: "Под описание подходит несколько студентов: %s\n", a...: strings.Join(e
3   } else {
2     fmt.Println(a...: "Студент с такими характеристиками в базе не найден.")
1   }
80 }

```

NORMAL lab-01 <01-graphs/1.2\_student\_expert/main.go > f main gj < 14 Bot 80:1 © 17:10

```

→ 1.2_student_expert git:(lab-01) x pwd
/Users/alex/itmo/algorithms-labs-2026/lab-01-graphs/1.2_student_expert
→ 1.2_student_expert git:(lab-01) x go fmt main.go
→ 1.2_student_expert git:(lab-01) x go build main.go
→ 1.2_student_expert git:(lab-01) x go run main.go
Система определения студентов
Студент мужского пола? (да/нет): нет
Студент курит? (да/нет): нет
Студент является брюнетом? (да/нет): нет
Студент учился в ИТМО в бакалавриате? (да/нет): нет
Студент занимается спортом? (да/нет): нет
У студента есть домашние животные? (да/нет): да
Это собака? (да/нет): да

Результат поиска:
Студент с такими характеристиками в базе не найден.
→ 1.2_student_expert git:(lab-01) x go run main.go
Система определения студентов
Студент мужского пола? (да/нет): нет
Студент курит? (да/нет): нет
Студент является брюнетом? (да/нет): нет
Студент учился в ИТМО в бакалавриате? (да/нет): да
Студент занимается спортом? (да/нет): да
У студента есть домашние животные? (да/нет): нет

Результат поиска:
Вы загадали студента: Фролова Кристина
→ 1.2_student_expert git:(lab-01) x |

```

### 1.3 Задание 3

#### Условие:

Составьте граф для задания №2. К какому типу относится данный граф?

#### Анализ структуры:

Логику работы нашей написанной поисковой программы можно представить как обычную развилку дорог или дерево вариантов.

Каждый вопрос в консоли это блок (вершина графа) с надписью, например, **Студент курит** или **Учился в ИТМО**. Из каждого блока выходят всего две стрелочки (ребра графа): одна означает **да**, а вторая означает **нет**. Когда пользователь вводит свой ответ, программа делает шаг по нужной стрелочке.

Компьютер не выделяет память, вместо этого обычный цикл просто идет по списку студентов и вычеркивает тех, кто не подошел под выбранную стрелочку.

Наша фишка с домашними животными работает здесь как короткий путь: если пользователь говорит, что зверей нет, программа понимает, что идти к вопросу про собаку нет никакого смысла, и просто перепрыгивает эту развилку. В самом низу этого дерева дорог нас ждут тупики, это конечные узлы (листья графа), где лежат конкретные имена ребят из нашей таблицы.

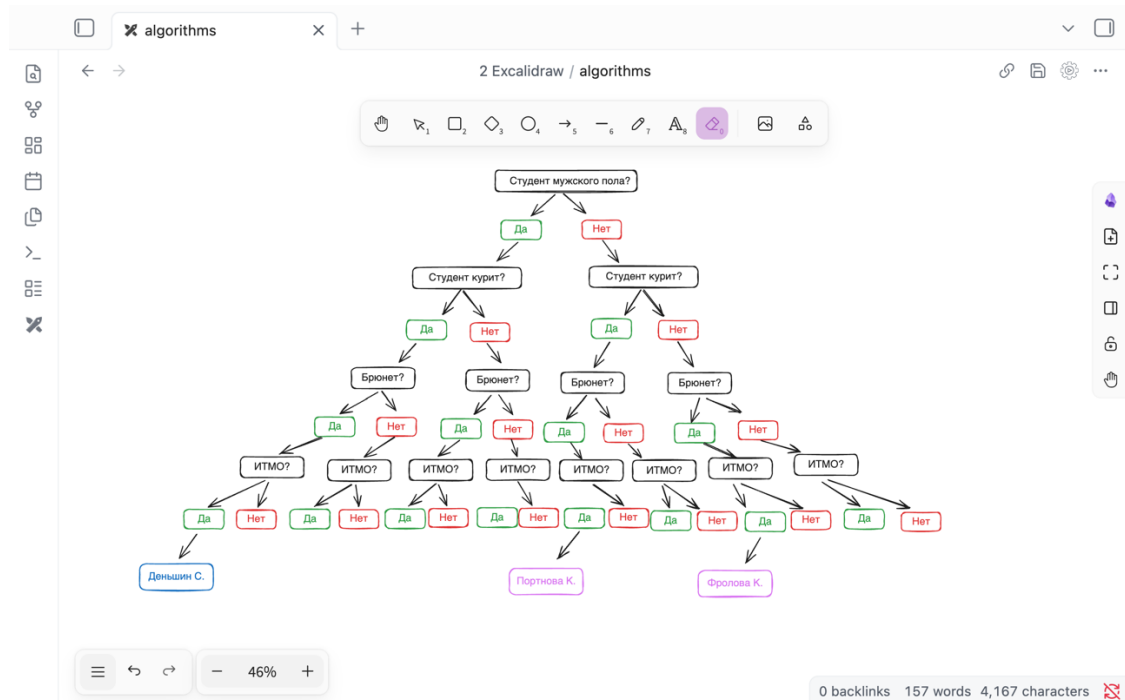
#### Определение графа:

Этот граф называется направленным ациклическим графом, а если говорить более простым языком, то это бинарное дерево решений.

Во первых, он является направленным, потому что у всех стрелочек есть четкое направление сверху вниз. Мы всегда движемся от первого вопроса к конкретному имени студента. Пойти по стрелочке в обратную сторону или снизу вверх компьютерная логика не позволяет.

Во вторых, граф является ациклическим, потому что в нем полностью отсутствуют круги или петли. Вы не можете застрять в этой программе и ходить бесконечно по кругу вокруг одних и тех же вопросов. Каждый ответ навсегда отрезает ненужную половину путей, поэтому мы гарантированно дойдем до конца и найдем студента за минимальное число вопросов.

### Схема (Дерево решений):



## 2 LEETCODE

### 2.1 Задача 1315. Sum of Nodes with Even-Valued Grandparent

**Accepted** 76 / 76 testcases passed  
Alex submitted at Sep 16, 2026 19:14

**Runtime**  
0 ms | Beats 100.00%

**Memory**  
9.12 MB | Beats 65.38%

```
9 func sumEvenGrandparent(root *TreeNode) int {
10     sum := 0
11
12     var dfs func(node *TreeNode, parent *TreeNode, grandparent
13         *TreeNode)
14     dfs = func(node *TreeNode, parent *TreeNode, grandparent
15         *TreeNode) {
16         if node == nil {
```

Testcase | Test Result

Case 1 | Case 2

Input

root =  
[6,7,8,2,7,1,3,9,null,1,4,null,null,null,5]

### 2.1 Задача 785. Is Graph Bipartite?

**Accepted** 87 / 87 testcases passed  
Alex submitted at Sep 16, 2026 19:25

**Runtime**  
0 ms | Beats 100.00%

**Memory**  
7.99 MB | Beats 96.52%

```
1 func isBipartite(graph [][]int) bool {
2     colors := make([]int, len(graph))
3
4     for i := 0; i < len(graph); i++ {
5         if colors[i] != 0 {
6             continue
7         }
8
9         queue := []int{i}
10        colors[i] = 1
11
12        for len(queue) > 0 {
```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

## ЗАКЛЮЧЕНИЕ

Эта лабораторная работа помогла понять, что знание базовых алгоритмов спасает от написания неоптимального кода, который бы тормозил на реальных проектах. Понимание теории графов и практики на платформе LeetCode дает возможность видеть логику за рамками простых условий, что позволяет находить самые эффективные инженерные решения при проектировании бэкенда.

### **СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ**

1. Бхаргава А. Грожаем алгоритмы. Второе издание, 2026.
2. Документация по языку программирования Go, ссылка: <https://go.dev/doc/>
3. Сайт с задачами LeetCode, ссылка: <https://leetcode.com/problemset/>